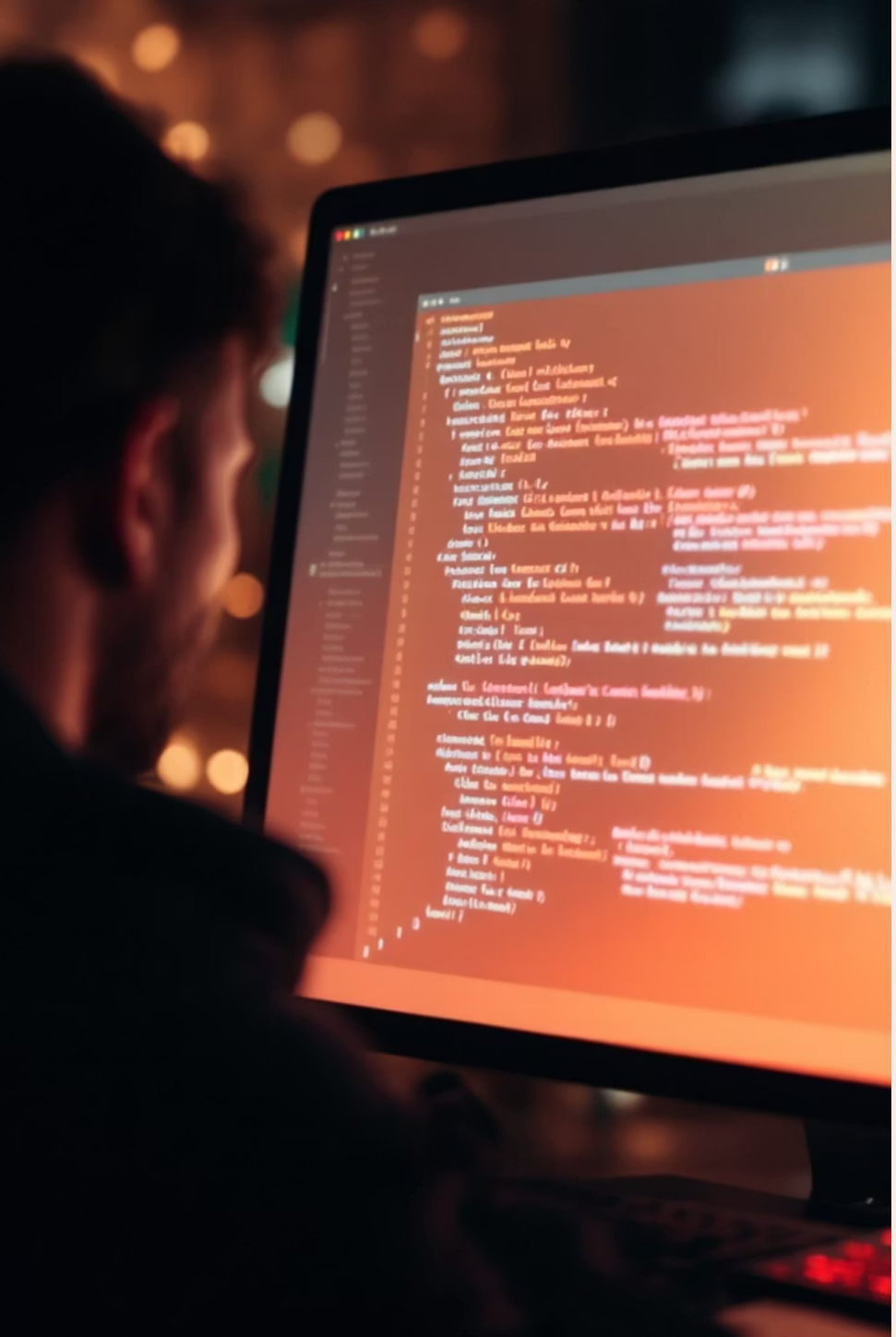


Python Nested Dictionaries: Organising Your Data Like a Pro

A beginner's guide to mastering hierarchical data organisation in Python

Made with GAMMA

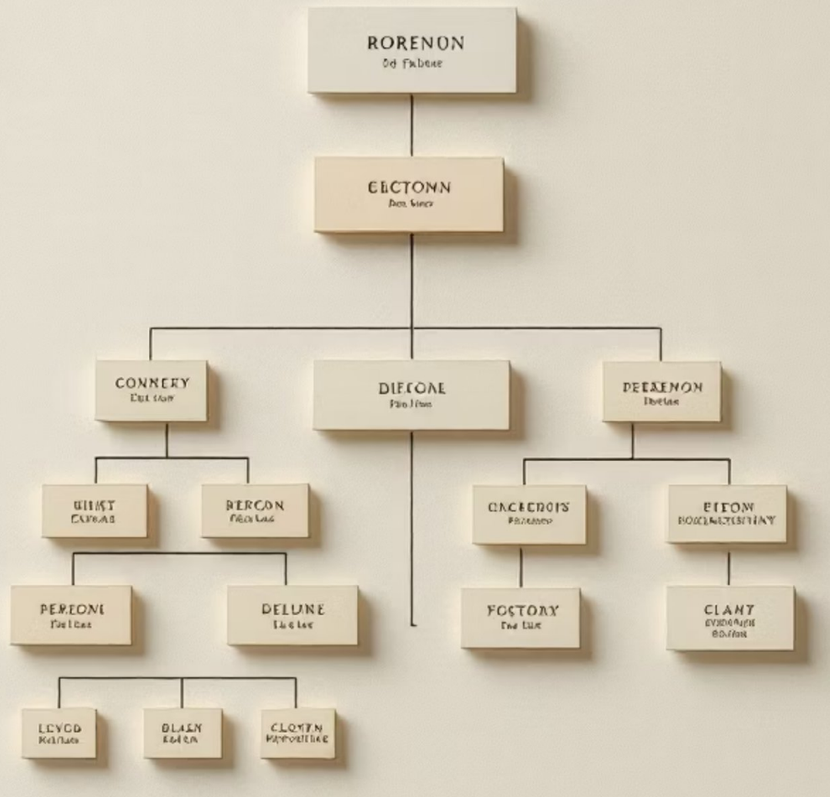
What's a Dictionary? (A Quick Recap)

Think of a dictionary like a real-world dictionary where each entry has two parts working together:

Key: The word you look up (e.g., "apple")
Value: The definition of that word (e.g., "a round fruit with firm, white flesh and a green, red, or yellow skin")

In Python, keys must be unique whilst values can be anything—numbers, strings, lists, or even other dictionaries!

Made with GAMMA



Introducing Nested Dictionaries: Dictionaries Inside Dictionaries!

What Is It?

A dictionary where the value itself is another dictionary, creating multiple levels of organisation

Perfect For

Organising complex, hierarchical data that naturally has multiple levels and relationships

Real Analogy

Like a family tree where each person's entry leads to another dictionary with their details

Made with GAMMA

Creating Your First Nested Dictionary

Example: Student Records

We want to store student names, their subjects, and the marks they got in each subject. Here's how:

```
students = {
    "Alice": {
        "Maths": 85,
        "Science": 92,
        "English": 78
    },
    "Bob": {
        "Maths": 90,
        "Science": 88,
        "English": 84
    },
    "Charlie": {
        "Maths": 75,
        "Science": 82,
        "English": 91
    }
}
```

Each student name is a key, and their value is another dictionary containing subject-wise marks!

Made with GAMMA

Accessing Data in Nested Dictionaries

Basic Access

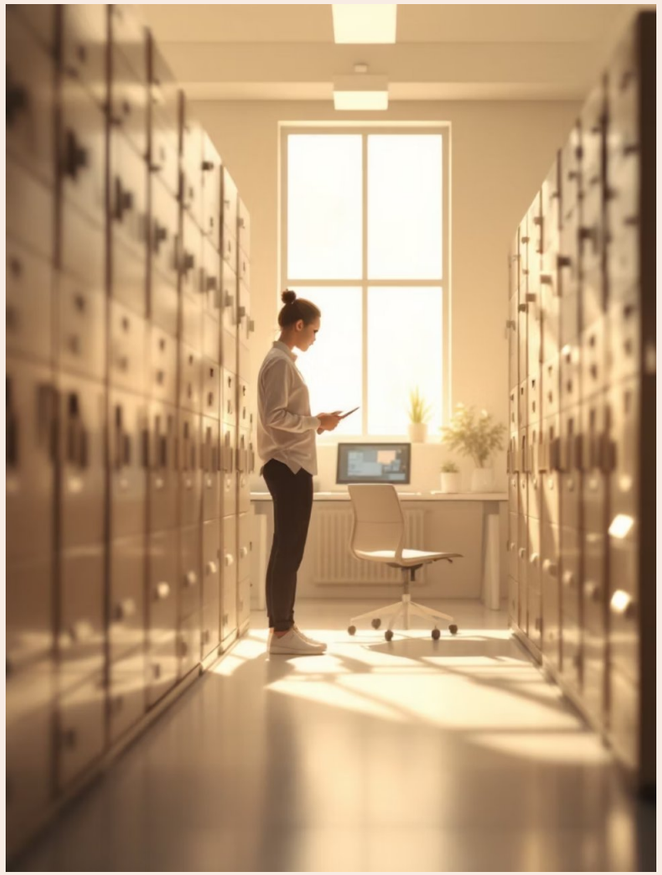
To access specific data, use multiple square brackets in sequence:

```
# Get Alice's Maths marks
alice_maths = students["Alice"]["Maths"]
print(alice_maths)  # Output: 85

# Get Bob's English marks
bob_english = students["Bob"]["English"]
print(bob_english)  # Output: 84
```

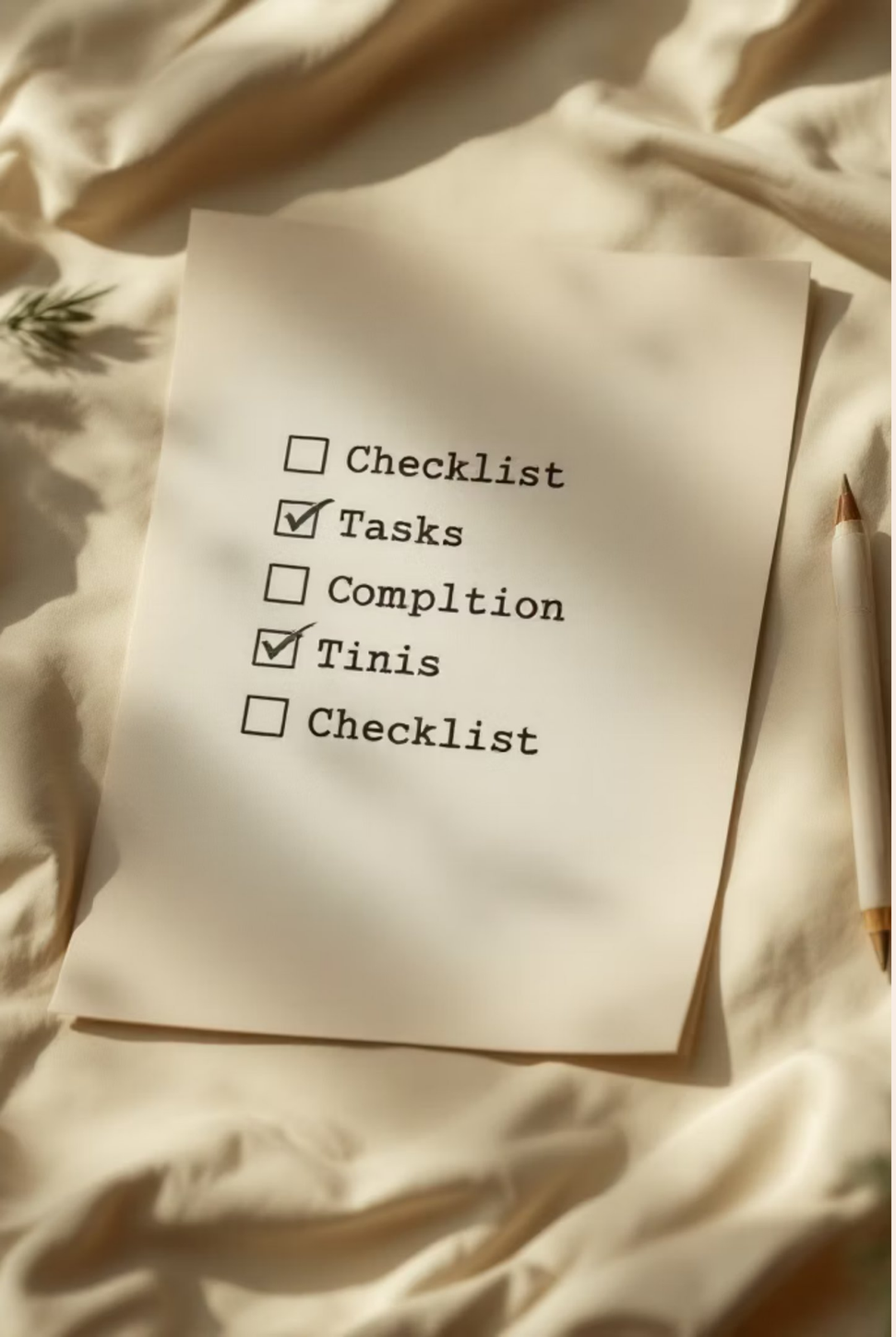
Adding New Data

```
# Add a new subject for Charlie
students["Charlie"]["History"] = 88
```



Pro Tip: Always check if a key exists before accessing it using the `get()` method or `in` operator to avoid errors!

Made with GAMMA



Common Operations & Practical Tips

01

Check if Key Exists

Use `in` operator: `"Maths" in students["Alice"]` returns `True`

02

Safe Access with get()

Use `get()` method: `students["Alice"].get("Art", 0)` returns `0` if "Art" doesn't exist

03

Iterate Through Data

Use nested loops to process all students and their marks efficiently

04

Update Existing Values

Simply assign new values: `students["Alice"]["Maths"] = 90`

Made with GAMMA

Key Takeaways

Hierarchical Organisation

Nested dictionaries let you organise data in multiple levels, perfect for representing real-world relationships like student records, employee details, or product categories

Flexible Structure

Values can be anything—strings, numbers, lists, or more dictionaries. This flexibility makes them incredibly powerful for complex data structures

Access with Care

Always verify keys exist before accessing nested data. Use `get()` method or try-except blocks to handle missing keys gracefully

Made with GAMMA